

Setting up moderation

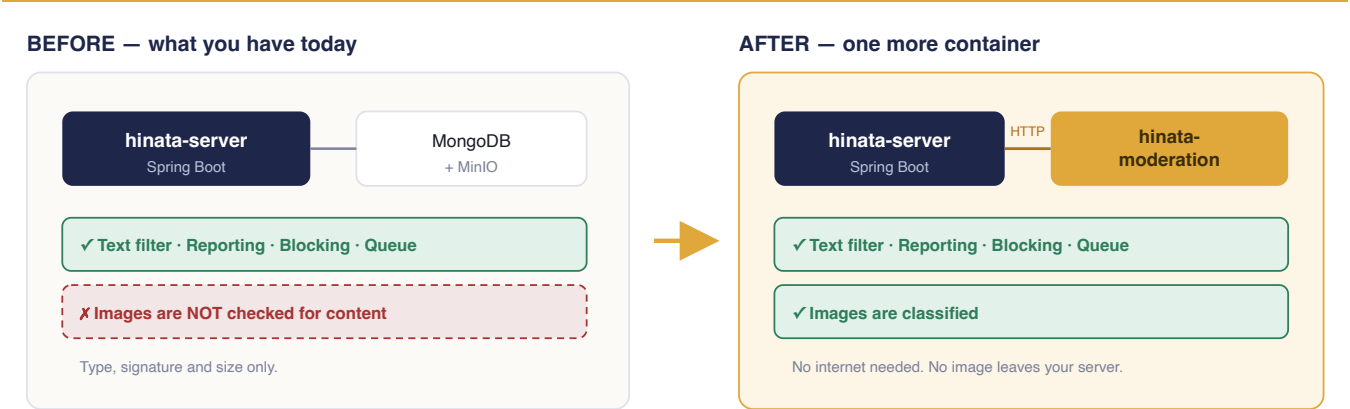
How `hinata-moderation` gets into a running Hinata installation — and how to check that it is actually doing something.

For anyone running their own Hinata instance · As of 6 August 2026 · applies from the server version that carries the moderation package

In one sentence

Hinata checks text out of the box; **it only checks images once you add a second container**. This guide adds it, connects it, and shows you how to tell that it is genuinely classifying rather than merely running.

What you already have — and what is missing



The sidecar runs on the same compose network. It deliberately publishes no port.

The classifier is *not* in the server image, because a ViT model plus its runtime weighs a few hundred megabytes — which every installation would otherwise carry, including the many that never receive an image.

What runs out of the box

Feature	Default	What it does
Text filter	on	German + English, over titles, descriptions, comments, articles, project and team names. No model, no network.
Reporting & blocking	on	Anyone can report and block content <i>and</i> people.
Moderation queue	on	Admin area → Moderation.
Image classification (NSFW)	off	Needs the sidecar. That is what this guide is for.

Hash matching (known material)	off	Needs access to one of the programmes — see step 5.
Voice notes	unchecked	Stored and served without ever being examined.
PDF / Office files	unchecked	Type and file signature are checked; the contents are not read.

Two things that are easy to misread

The text filter is deliberately reluctant. Long text is put in front of a human rather than refused — a wrongly refused bug report does not get rewritten, it gets abandoned. Short fields that everyone sees, like a project name, are refused outright.

No setting can weaken two categories: child sexual abuse material and malware. There is deliberately no control for them.

1 Add the sidecar

Put the file next to your `docker-compose.yml` (or merge it into an override you already have). A ready-made version ships at `hinata-moderation/docker-compose.override.yml`.

```
# docker-compose.override.yml
services:
  hinata-moderation:
    image: ghcr.io/hinata-platform/hinata-moderation:1.0.0
    restart: unless-stopped
    networks: [hinata-moderation]

    # No `ports:` – deliberately. Nobody outside has any business posting
    # here, and a published port is how a classifier with no auth ends up
    # reachable on a NAS that also forwards 80 and 443.
    security_opt: [ "no-new-privileges:true" ]
    cap_drop: [ ALL ]
    read_only: true
    tmpfs: [ "/tmp:size=64m,mode=1777" ]

    environment:
      MAX_BYTES: "16777216"          # 16 MB
      MAX_CONCURRENT: "1"
      MAX_QUEUED: "8"
      # INTRA_OP_THREADS: "2"        # auf einer NAS setzen: ONNX nimmt sonst alle Kerne

    deploy:
      resources:
        limits: { memory: 1g }
        reservations: { memory: 512m }

  hinata-server:
    networks: [hinata-moderation]

networks:
  hinata-moderation:
```

Check once before restarting

The fragment *adds* a network to `hinata-server`. Whether your base file keeps its existing ones is answered by:

```
docker compose config | grep -A6 "hinata-server:"
```

A server that loses its database network to a merge surprise is a much worse afternoon than a classifier that never got wired up.

```
docker compose up -d hinata-moderation
docker compose ps hinata-moderation    # must report "healthy"
```

2 Connect the server — from the admin area

This needs **no** environment variable and no server restart. The endpoint lives in the admin area and takes effect immediately.

Admin area > Moderation

Content checking

Moderation active ☒

Classify images ☒

⚠ No image classifier installed
Uploaded images are not being checked.

Image classifier

Address of the moderation sidecar

Timeout

Schematic of the panel (derived from the widget code, not a screenshot). The warning sits directly above the field that resolves it.

Use the service name from your compose file as the address, not `localhost` :

```
http://hinata-moderation:8081
```

Still settable by environment variable if you prefer (`HINATA_MODERATION_IMAGE_ENDPOINT`). What the admin area holds wins — as with every other moderation setting.

3 Verify that it is genuinely classifying

The step that matters most

"The container is running" and "images are being checked" are two different facts. That confusion is exactly why the admin area now carries a status line of its own.

ACTIVE Images are being checked. nothing to do	NOT_CONFIGURED No classifier configured. → step 2	CONFIGURED_UNAVAILABLE Configured, but not responding. → Troubleshooting	DISABLED_BY_POLICY You switched it off. deliberate
---	--	---	---

The four states. Only **ACTIVE** means images are actually being looked at.

Three independent ways to confirm it:

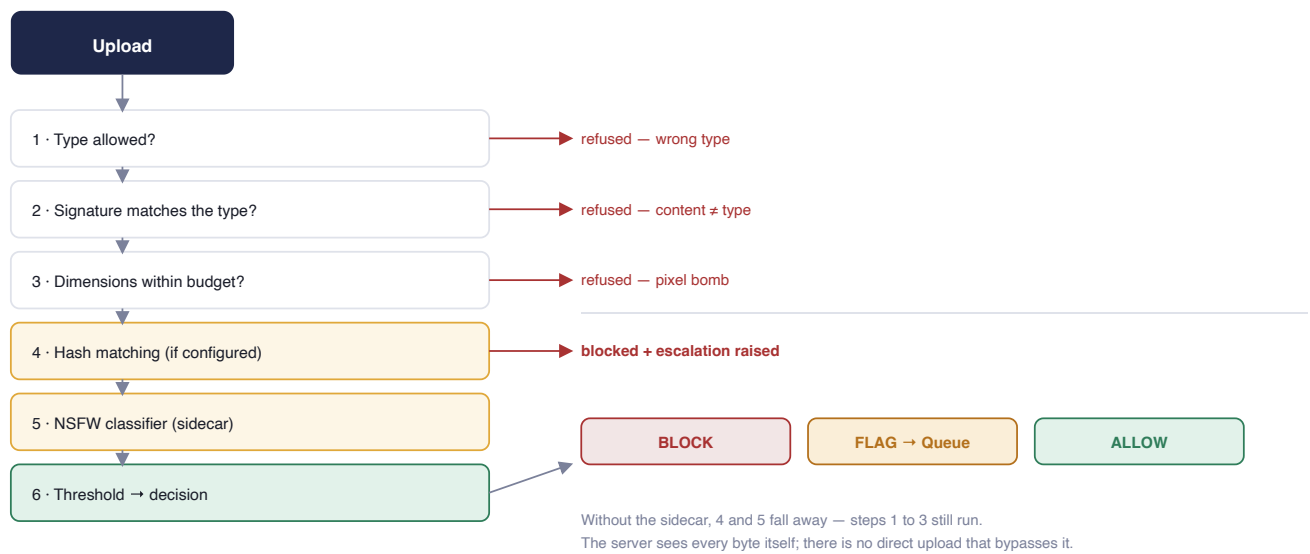
1. **Admin area → Moderation.** The line under the *classify images* switch names one of the four states.
2. **The server log at startup.** If image moderation is on and no classifier is reachable, there is a `WARN` line naming the missing key.
3. **Straight at the sidecar** (from a container on the same network):

```
docker compose exec hinata-server \  
curl -s http://hinata-moderation:8081/v1/health
```

Make it a decision

If you do *not* want image checking, switch it off in the admin area. The panel then says "off" instead of implying a check that is not happening — and nobody relies on it later.

What happens to an uploaded image



Step 3 protects the sidcar too: a pixel bomb is refused before anything decodes it.

4 Set up the escalation webhook

This is how you find out *immediately* that something serious was reported, rather than the next time somebody opens the admin area. Admin area → Moderation → *Escalation*:

Field	What it is
URL	Where the notice goes — alerting, a chat webhook, a ticket inbox.
Secret	Signs the message (X-Hinata-Signature , HMAC-SHA256). Required — a URL without a secret is refused on save, and the adapter will not deliver unsigned. A "content frozen" notice its recipient cannot attribute is worse than none. Never echoed back; an empty field means "keep the stored one", not "clear it".
Timeout / attempts	Delivery with a bounded retry. Failures are audited, never swallowed.

The payload is deliberately boring

It carries a record id, a category, a surface and a timestamp. **No content, no file name, no bytes.** That is why you can safely point it at a chat channel — the alert travels further than the content ever should.

5 Apply for hash matching (optional, takes weeks)

Hinata has a slot for this and **ships no implementation, and never will**. These programmes vet the *operator organisation*, not the software; their terms forbid passing credentials on; and their hash lists live on their servers precisely so nobody holds a local copy to test evasion against.

Worth doing above all if your instance accepts content from outside your organisation — an inbound e-mail address that creates tickets, for example.

- Microsoft **PhotoDNA Cloud Service** — microsoft.com/photodna
- **Thorn Safer** — safer.io
- Google **Child Safety Toolkit** — protectingchildren.google
- Cloudflare **CSAM Scanning Tool** — only inspects what passes through Cloudflare's cache; private attachments streamed from your own object storage may be invisible to it. Confirm coverage before treating it as protection.

The credentials are then yours and are not transferable. The integration itself is a small adapter against the existing port.

When the escalation fires

Do not open the content to check.

This is the most important line in the guide. Looking is not verification — it does not make the report more accurate, and for one category it is itself the harm. The software will not show it to you either: frozen content is unreachable for administrators as well.



Freezing hides, it does not delete. Evidence is preserved; access ends — for admins too.

Hinata reports to nobody on your behalf. It does not know your jurisdiction and is not the reporting party.

Before you go live: write this chain down and *name the person*. One named person, with a deputy. "The admins" is not an answer at 23:00 on a Friday.

Troubleshooting

Symptom	Cause and fix
NOT_CONFIGURED even though the container is running	The endpoint is empty or wrong. Use the service name, not <code>localhost</code> . Check with: <code>docker compose exec hinata-server curl -s http://hinata-moderation:8081/v1/health</code>
CONFIGURED_UNAVAILABLE	Address reachable, but the sidecar reports itself unhealthy — usually a failed model load. <code>docker compose logs hinata-moderation</code>
Uploads suddenly slow	The sidecar runs one worker. On a NAS, set <code>INTRA_OP_THREADS</code> so ONNX does not take every core and compete with the database.
Images are abgelehnt, die harmlos sind	Thresholds are in the admin area. The "flag long text only" control affects text, not images.
The sidecar is OOM-killed	The memory limit is the point — this container holds the attacker-supplied pixel buffers. Better it than the database beside it.

What is still not checked afterwards

So you do not rely on something that is not happening:

Area	Status
Voice notes	Not transcribed and not checked.
PDF and Office contents	Type and signature only, no text content.
Video	No frame analysis.
Statement of reasons on a freeze	Not generated automatically. If you need one, it is a manual step in your procedure.
Notifications already sent	A freeze cannot recall what already went out by mail or push. It only stops what would follow.

And the legal side?

Deliberately not here, but as *questions* in `OPERATORS.md` — because the answers depend on how your instance is configured, and because the field moves quickly. A document that states them as settled is wrong for some operators immediately and for all of them eventually.

One sentence always holds: whoever hosts carries the obligations. That is you, not the Hinata project.